

Full Length Article

Generative binary memory: Pseudo-Replay class-Incremental learning on binarized embeddings^{*}Yanis Basso-Bert^{ID}, William Guicquero^{ID}, Anca Molnos^{ID}*, Romain Lemaire^{ID}, Antoine Dupret^{ID}

Univ. Grenoble Alpes, CEA, List, Grenoble, F-38000, France

ARTICLE INFO

Keywords:

Continual learning
Binary neural network
Bernoulli mixture model
Quantization
Class-incremental learning

ABSTRACT

In dynamic environments where new concepts continuously emerge, Deep Neural Networks (DNNs) must adapt by learning new classes while retaining previously acquired ones. This challenge is addressed by Class-Incremental Learning (CIL). This paper introduces Generative Binary Memory (GBM), a novel CIL pseudo-replay approach which generates synthetic binary pseudo-exemplars. Relying on Bernoulli Mixture Models (BMMs), GBM effectively models the multi-modal characteristics of class distributions, in a latent, binary space. With a specifically-designed feature binarizer, our approach applies to any conventional DNN. GBM also natively supports Binary Neural Networks (BNNs) for highly-constrained model sizes in embedded systems. The experimental results demonstrate that GBM achieves higher than state-of-the-art average accuracy on CIFAR100 (+2.9%) and TinyImageNet (+1.5%) for a ResNet-18 equipped with our binarizer. GBM also outperforms emerging CIL methods for BNNs, with +3.1% in final accuracy and $\times 4.7$ memory reduction, on CORE50.

1. Introduction

Many real-world applications face changing contexts, with continually emerging new concepts and categories (Hadsell et al., 2020). A Deep Neural Network (DNN) deployed in such dynamic environments should thus possess the ability to learn new classes, while preserving prior knowledge of past classes, a learning paradigm referred as Class-Incremental Learning (CIL) (Kirkpatrick et al., 2017). However, without any precautions, DNNs tend to forget previously learned patterns when being fine-tuned on new data, a phenomenon known as catastrophic forgetting (McCloskey & Cohen, 1989). Numerous methods now exist (Wang et al., 2024) to effectively learn from a new set of classes, also denoted as a new task, without forgetting past classes.

Among the available methods, exemplars-based approaches have shown superior performance by storing and replaying a small portion of past examples, i.e., exemplars, during training (Wang et al., 2024). However, this advantage comes at the cost of increased memory requirements. Storing even a small subset of prior examples demands additional memory resources. This poses a significant challenge to CIL deployment on embedded systems, where memory is a scarce resource (Ravaglia et al., 2021). To address this, considerable efforts have focused on reduc-

ing memory size, primarily by compressing the stored exemplars (Wang et al., 2021) or storing exemplars in an embedding space, method also known as latent replay (LR) (Pellegrini et al., 2020).

More recently, LR is extended to Binary Neural Networks (BNNs) (Hubara et al., 2016). BNNs are a valuable approach, as they significantly reduce memory demands, computational complexity, and energy consumption through binary weights and activations (Yuan & Agaian, 2023). Furthermore, LR on BNNs has only a small memory overhead because of the binary nature of the embedding (Basso-Bert et al., 2024; Lorenzo et al., 2024).

This work takes a step further by generating synthetic, binary, pseudo-exemplars on-the-fly, rather than replaying stored binary exemplars. A generative model offers the potential to significantly reduce memory footprint and to avoid overfitting by interpolating between past training data. Existing exemplar-free CIL methods (e.g., EFCIL (Petit et al., 2023)) compute and store only a representative prototype for each past class. Synthetic pseudo-exemplars are generated around prototypes by data augmentation heuristics. Building upon EFCIL, we investigate the utilisation of binary embeddings, as briefly presented in Fig. 1.

As illustrated in Fig. 1a), in a VGG-like BNN we observe that binary features (with values in $\{-1, +1\}$) tend to cluster by class, with some

^{*} This work is part of the IPCEI Microelectronics and Connectivity and was supported by the French Public Authorities within the frame of France 2030.

^{*} Corresponding author.

E-mail addresses: ybassobert@gmail.com (Y. Basso-Bert), william.guicquero@cea.fr (W. Guicquero), anca.molnos@cea.fr (A. Molnos), romain.lemaire@cea.fr (R. Lemaire), antoine.dupret@cea.fr (A. Dupret).

<https://doi.org/10.1016/j.neunet.2026.108884>

Received 20 March 2025; Received in revised form 14 October 2025; Accepted 20 March 2026

Available online 21 March 2026

0893-6080/© 2026 The Authors. Published by Elsevier Ltd. This is an open access article under the CC BY license (<http://creativecommons.org/licenses/by/4.0/>).

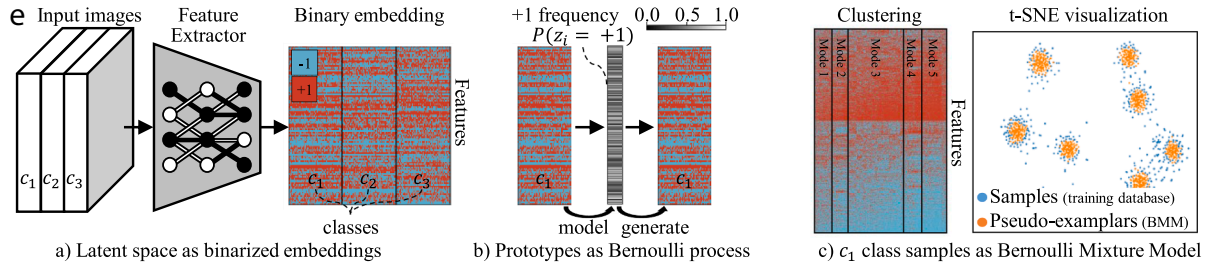


Fig. 1. Motivations of our pseudo-replay CIL approach based on Bernoulli Mixture Model (BMM). a) Illustration of per-class features correlation on binary embedding. b) Illustration of a single prototype encoding and pseudo-exemplars generation. Prototype features is computed as the $+1$'s frequency in a class embedding. Pseudo-exemplars are generated from the prototype as a Bernoulli process. c) Qualitative investigation of the binary distribution: (left) apparition of modes when reorganising samples and features with hierarchical clustering, (right) t-SNE visualization (Van der Maaten & Hinton, 2008) of training samples and pseudo-exemplars generated from a BMM.

features having higher flip rates. The $+1$'s frequency for each feature may constitute a representative prototype of a class' binary embedding distribution. The class distributions can then be synthesized by Bernoulli processes. Moreover, we observed that multiple modes appear when applying hierarchical clustering (Nielsen, 2016) to class embedding (Fig. 1c), suggesting that a single prototype may not capture the full diversity of a class. To address this, the Bernoulli Mixture Model (BMM) offers a statistical generative framework to model binary vectors with multi-modal characteristics (Fig. 1c). This paper brings the following contributions:

- Generative Binary Memory (GBM), a pseudo-replay CIL method on binarized embeddings, based on BMMs.
- Dedicated embedding binarizers making our GBM compatible with any features extractor and DNN.
- Experimental results on a ResNet-18 (He et al., 2016) demonstrating GBM improves state-of-the-art performance in comparison with other prototypes methods, e.g., by $+1.5\%$ against FeTril (Petit et al., 2023) on the challenging dataset of TinyImageNet with 20 incremental tasks.
- Experimental results on GBM with BNNs, managing CIL under a limited memory budget dedicated to tiny DNN models, outperforming emerging work (Basso-Bert et al., 2024; Lorenzo et al., 2024) with $+3.1\%$ in final accuracy and $\times 4.7$ memory reduction, on the CORE50 benchmark.

2. Related work

Continual Learning. Among the continual learning problems in the literature (Wang et al., 2024), our focus is on Class Incremental Learning, CIL (Rebuffi et al., 2017), whose goal is to learn a unified classifier without relying on task labels. CIL inherently involves a plasticity-stability trade-off, where the model must remain flexible to learn new tasks and stable enough to preserve previously acquired knowledge. Various methods exist, including architectural-based approaches (Yan et al., 2021), regularization-based techniques, like Elastic Weight Consolidation (EWC) (Kirkpatrick et al., 2017) and Learning without Forgetting (LwF) (Li & Hoiem, 2017), and replay-based strategies (Rebuffi et al., 2017). Replay methods, such as iCaRL (Rebuffi et al., 2017), EEIL (Castro et al., 2018), and LUCIR (Hou et al., 2019), are effective but introduce the additional memory overhead related to past tasks exemplars. LR methods (Pellegrini et al., 2020) address the memory issues, storing latent features instead of raw data.

Furthermore, prototype-based methods rely on class centroids and distributions modeling to generate synthetic data. For instance, PASS (Zhu et al., 2021b) uses isotropic augmentation based on the trace of the covariance matrix, while IL2A (Zhu et al., 2021a) accounts for feature inter-dependencies estimations using a multivariate Gaussian approximation. More recently, FeTril (Petit et al., 2023) augments the

distribution of old classes based on new classes statistics. Knowledge distillation (Gou et al., 2021) can also cap drift from previously learned distributions (Zhu et al., 2021a,b).

Like FeTril, we address a system with a fixed feature extractor to ensure a stable representation, while retraining only the classifier. Our proposed method further differentiates from other prototype-based approaches by (1) working on categorical binary features rather than continuous (2) handling the multi-modality of class distributions using a mixture model, and (3) applying prototype quantization.

Bernoulli Mixture Model in CIL. BMMs have been widely studied to model multidimensional categorical data (Bishop & Nasrabadi, 2006; Juan et al., 2004; Lazarsfeld & Henry, 1968; Li et al., 2016), with applications in clustering (Najafi et al., 2019), classification (Alalyan et al., 2020; Gimenez et al., 2014), and dimensionality reduction (Saeed et al., 2013). More recently, BMMs are combined with deep neural networks, particularly for parameterizing priors in variational autoencoders (Loaiza-Ganem & Cunningham, 2019; Wu et al., 2019), image retrieval (Amato et al., 2016, 2018) and multi-instance learning (Yang & Wu, 2020). BMMs have not been yet applied to CIL, unlike Gaussian Mixture Models (GMMs) (Bishop & Nasrabadi, 2006). For example, Gaussian Mixture Replay (Pfülb & Gepperth, 2021) models and generates input images using GMMs, however it limits to simple datasets such as MNIST (LeCun et al., 1998). Furthermore, MIX (Korycki & Krawczyk, 2024) learns GMMs and uses them as classifiers, but performs well only when the model is pre-trained on the whole dataset.

Binary Neural Network in CIL. BNNs have binary weights and activations (e.g., $\{-1, +1\}$) (Yuan & Agaian, 2023) and thus all scalar products are replaced with more efficient XNOR logic gates and bit-count operations (Hubara et al., 2016). As a result, BNNs drastically reduce memory footprint, and energy consumption, making them highly efficient in terms of hardware implementation. Despite their non-differentiable nature, BNNs training through back-propagation is made possible by the Straight-Through Estimator (STE) (Bengio et al., 2013) in a Quantization-Aware Training (QAT) approach (Kulkarni et al., 2022). Nevertheless, BNNs' performance does not yet match the one of the corresponding conventional, real-valued networks. A way to narrow this gap is to keep some layers in floating-point precision (Bannink et al., 2021; Liu et al., 2020, 2018), leading to Hybrid BNNs. Another branch of research focuses on optimizing BNNs that entirely rely on binary arithmetic (Basso-Bert et al., 2024; Ma et al., 2024), i.e., Fully BNNs, to enable disruptive hardware implementations.

Recent work also explores CIL for BNNs. Synaptic metaplasticity (Laborieux et al., 2021) introduces a regularization function that computes weights' importance based on weights' magnitude during QAT, however is limited to simple datasets. LR on binary embedding is proposed in Lorenzo et al. (2024), taking advantage of BNNs' compression. Finally, Basso-Bert et al. (2024) explores replay mechanisms in fully binary networks, showing the suitability of both experience and LR for

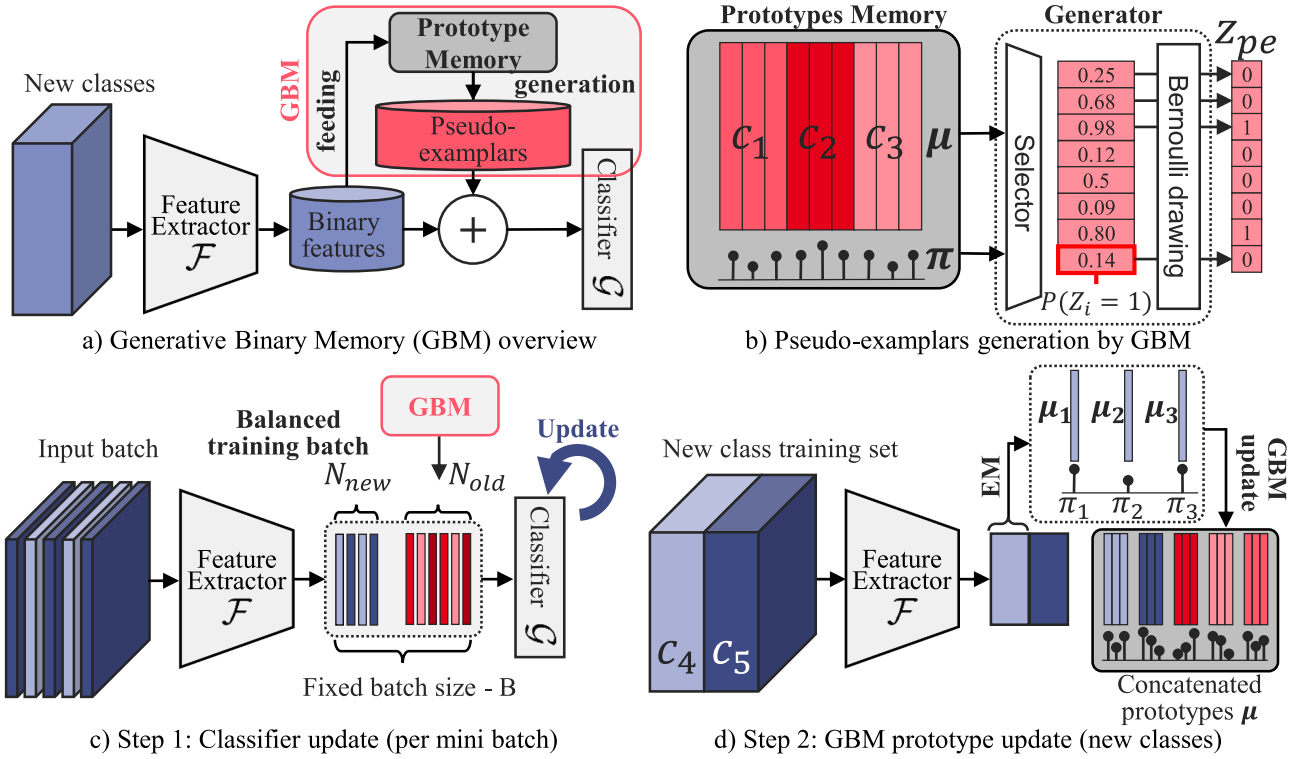


Fig. 2. Our proposed GBM method for Pseudo Replay on Binary embedding: a) overview of our framework, b) generative procedure from the prototype memory, c) Classifier update on class-balanced batch between new class samples and past class pseudo-exemplars, and d) GBM update with per-class BMM modeling with EM algorithm and prototype concatenation in the memory.

ultra-low power devices. This work goes beyond replay methods by generating binary pseudo-exemplars based on intra-class correlations, to increase performance and further reduce CIL memory needs.

3. Generative binary memory

CIL problem formulation. The CIL problem involves learning a sequence of classification tasks, starting from an initial task, with training dataset D_0 , followed by T incremental tasks, with training datasets $D_1, \dots, D_T$. $D_t = \{(x_{t,i}, y_{t,i})\}_{i=1}^{N_t}$ is the training dataset that the model receives at task t . D_t consists in N_t labeled samples, $x_{t,i}, y_{t,i} \in C_t$ denotes the class labels, and C_t is the set of classes for task t . There is no intersection between the classes of different tasks, meaning $C_i \cap C_j = \emptyset$ for $i \neq j$. Typically, at task t , the model is updated only on the training examples from the current task, D_t and evaluated on the classification problem containing all seen classes $\bigcup_{i=0}^t C_i$.

System overview. We consider a neural network composed of a feature extractor F and a classifier G . This neural network is extended with our Generative Binary Memory (GBM), as depicted in Fig. 2a). The GBM includes a prototype memory that: (1) can be updated when new classes appear, and (2) can generate synthetic pseudo-exemplars (Fig. 2b)). F is trained during the initial task and then frozen. For each incremental task, the system is updated in two steps. First, G is re-trained on new training samples mixed with generated pseudo-exemplars associated to old classes (Fig. 2c)). Second, the GBM is updated with prototypes computed on the new classes training set. Prototypes and their mixing coefficients are computed per-class as BMM with an EM algorithm (Fig. 2d)).

In what follows, Section 3.1 details how prototypes are computed, given the binary features. Section 3.2 then describes the prototype memory update and the process of data generation from prototypes. Finally, while our method is motivated by CIL on BNNs, we extend its applicability to any neural network; for this two embedding binarizer blocks are proposed in Section 3.3.

3.1. Multi-prototype encoding with BMM

At a given task t , GBM computes a multi-prototype representation of the N training samples, $X = \{x_{t,i} | y_{t,i} = c\}$ of a given class $c \in C_t$. Let $Z = F(X)$ be the N latent binary embedding set with dimension D , i.e. $Z = \{z_i\}_{i=1}^N \in \{0, 1\}^{N \times D}$. Under the assumption that Z is generated by $K \in \mathbb{N}$ underlying Bernoulli processes, we model the distribution with a BMM parameterized by $\Phi = (\mu, \pi)$, where $\mu = \{\mu_k\}_{k=1}^K \in [0, 1]^{K \times D}$ denotes the K Bernoulli probability vectors, namely prototypes, and $\pi = \{\pi_k\}_{k=1}^K \in [0, 1]^{K \times 1}$ denotes the corresponding mixing coefficients, subject to $\sum_{k=1}^K \pi_k = 1$.

An EM algorithm (Bishop & Nasrabadi, 2006) estimates the parameters Φ from Z . The EM algorithm iteratively refines Φ through two main steps: (1) an expectation step (E-step), and (2) a maximization step (M-step). In the E-step (Eq. 1), the responsibilities $\gamma_{i,k}$, i.e., the probability that a binary embedding z_i was generated by component k , are defined as:

$$\gamma_{i,k} = \frac{\pi_k \prod_{j=1}^D \mu_{k,j}^{z_{i,j}} (1 - \mu_{k,j})^{1-z_{i,j}}}{\sum_{l=1}^K \pi_l \prod_{j=1}^D \mu_{l,j}^{z_{i,j}} (1 - \mu_{l,j})^{1-z_{i,j}}} \quad (1)$$

In the M-step (Eq. 2), the model parameters μ and π are updated based on the computed responsibilities:

$$\pi_k = \frac{1}{N} \sum_{i=1}^N \gamma_{i,k} \quad \mu_{k,j} = \frac{\sum_{i=1}^N \gamma_{i,k} z_{i,j}}{\sum_{i=1}^N \gamma_{i,k}} \quad (2)$$

The BMM log-likelihood at iteration s of the EM-algo $\ell(\Phi^{(s)})$ is further defined in Eq. 3 as:

$$\ell(\Phi^{(s)}) = \sum_{i=1}^N \sum_{k=1}^K \gamma_{i,k} \log \left(\pi_k \prod_{j=1}^D \mu_{k,j}^{z_{i,j}} (1 - \mu_{k,j})^{1-z_{i,j}} \right) \quad (3)$$

The algorithm stops when the relative change $|\ell(\Phi^{(s)}) - \ell(\Phi^{(s-1)})|/|\ell(\Phi^{(s)})|$, falls below a predefined threshold ϵ .

Initialization. Initialization is critical for BMM to effectively converge and prevent “pathological cases” (Juan et al., 2004) such as prototype degeneration, i.e., when a prototype overfits on one single training point. We therefore initialize each prototype parameter μ by calculating the centroid of Z and then perturbing each μ_k around this centroid with a per-feature standard deviation. The mixing coefficients π are initialized and fixed at $\pi_k = \frac{1}{K}$, and not trainable, unlike in Eq. 3. This prevents prototype degeneration by encouraging a balanced prototype attribution during the E-step. Section 4 includes an experimental investigation of these BMM initialization choices. To further improve the initialization, we perform N_{init} warm-up rounds with different initializations, running the EM algorithm for N_{iter} iterations each. The initialization that yields the highest log-likelihood is selected, and then fully optimized up to the stopping criterion.

Post-EM prototypes quantization. While prototype methods are often described as memory-free (Zhu et al., 2021b), storing prototypes still requires memory, as highlighted in Petit et al. (2023). To be more efficient than conventional methods that store latent binary samples, GBM prototypes can be uniformly quantized (on q bits). Section 5.2 explores this, down to a prototype memory footprint close to the one of a few binary exemplars.

3.2. GBM Update and pseudo-exemplars generation

A BMM, $\Phi_{new} = (\mu_{new}, \pi_{new})$, is computed for each new encountered class. The GBM is updated by appending the newly computed prototypes μ_{new} to the existing set of prototypes μ , and the mixing coefficients π_{new} to π . To ensure that the concatenated π accurately represents the drawing probability across all seen classes, π is normalized based on the effective number of training samples per class.

During the classifier update on a new class, GBM generates pseudo-exemplars at each training batch. A pseudo-exemplar (z_{pe}, y_{pe}) is generated by first selecting a Bernoulli process μ_i in μ with selection probability π . Then a realization is drawn from the Bernoulli process parameterized by μ_i (Fig. 2b)). Within the fixed training batch size B , the number of new training data N_{new} and pseudo-exemplars N_{old} is set proportionally to the number of new n_{new} and previous classes n_{old} . This ensures an equal representation of all seen classes in batches (Eq. 4), to circumvent a possible imbalance dataset (Fig. 2c)).

$$N_{new} = \frac{B \times n_{new}}{n_{old} + n_{new}} \quad N_{old} = \frac{B \times n_{old}}{n_{old} + n_{new}} \quad (4)$$

3.3. Feature embedding binarization

GBM can be included in non-binary, conventional neural networks by adding an embedding binarizer and slightly adapting the training protocol of the initial task (Fig. 3). Two approaches are possible: (1) enforcing F to output directly a binary vector on its last layer; (2) coding learned full precision features into a binary vector.

Heaviside binarizer. For the first approach we propose to extend F with a point-wise convolution with $f \times D$ filters, followed by an Heaviside step activation for binary quantization. We train the model using the Straight-Through Estimator (STE) (Bengio et al., 2013) to compute the gradients in the Heaviside’s backward computation, to learn a binary feature representation. The binary embedding size is controlled by a factor f , as investigated in Section 4.

Thermometer binarizer. For the second approach, a thermometer representation (Buckman et al., 2018) encodes the real-valued D -dimensional embedding. A thermometer code represents an integer number on p bits, where the number of consecutive ones corresponds to the encoded value. Assuming that features values are within $[0, 1]$, we can uniformly quantize the interval $[0, 1]$ on p values and concatenate the associated thermocodes in one unique $p \times D$ -dimensional binary vector. Eq. 5 describes the thermode transformation $\hat{z}$ of a embedding vector

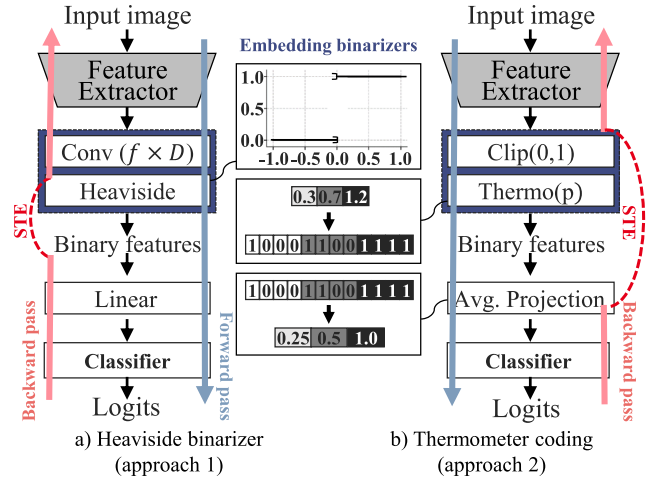


Fig. 3. The 2 proposed approaches for embedding binarization.

z_i refers to the i -th feature.

$$\hat{z}_{p \times i+j} = \begin{cases} 1 & \text{if } j \times \lfloor \text{clip}(z_i) \times p \rfloor \geq 1 \\ 0 & \text{otherwise} \end{cases} \quad (5)$$

Features need to be converted back in the real-valued domain, before being fed to the classifier. This conversion is done by averaging each thermocode segment (Eq. 6).

$$z_i = \frac{1}{p} \sum_{j=0}^{p-1} \hat{z}_{p \times i+j} \quad (6)$$

Since feature values may not be between 0 and 1, the output of F needs to be clipped in $[0, 1]$. However, to avoid too much activation saturation, a second training phase is performed with an STE from F ’s outputs to G ’s inputs. This way, F can learn to adjust its output dynamic range to the saturation and thermometer coding.

4. Evaluation on resnet architecture

This section reports an evaluation of our proposed approach using a conventional, real-valued ResNet architecture. The ResNet topology, with no pre-training, serves as the feature extractor, which is followed by a embedding binarizers, as presented in Fig. 3. This testbench allows for a fair comparison with state-of-the-art methods in CIL, as well as for an assessment of the generalizability of our technique, and the test of its robustness across various hyperparameters. Specifically, ResNet-18 (He et al., 2016) is used as F , which enables us to evaluate the applicability of GBM for pseudo-replay on rich features. The F terminates at the output of the average-pooling layer with an output dimension of $D = 512$. The classifier G consists of a linear projection.

4.1. Evaluation setup

Datasets and incremental setting. GBM is evaluated on two common datasets in CIL, in a standardized benchmark: CIFAR100 (Krizhevsky & Hinton, 2009) with 100 classes and TinyImageNet (Le & Yang, 2015) with 200 classes, using a fixed random order for classes. An initial task dataset D_0 , which consists of half of the classes, is considered for the pre-training of the entire model (i.e., the feature extractor with binarized outputs in the case of GBM, along with the classifier). The initial task performance is detailed in Table 2. The other half of the classes are divided across the incremental tasks. We study CIL in the three commonly reported settings, with $T = 5, 10$, and 20 classes. Specifically, for $T = 20$ on CIFAR100, the initial task includes 40 classes instead of 50.

Table 1

GBM average incremental accuracy compared to replay methods ($E = 20$ exemplars per class), memory-free regularization methods and prototype methods. Best GBM^T and GBM^H configurations from Table 2 are reported. **Best results**, second best results. Please note that this comparison with state-of-the-art techniques considers full floating-point models for prior work, while our GBM approach enables the integration of binarized embeddings with even a slight improvement in accuracy.

Category	Method	CIFAR-100			TinyImageNet		
		$T =$	5	10	20	5	10
Upper bound	Joint	77.0			62.4		
Replay $E = 20$	iCaRL	51.1	48.7	44.4	34.6	31.2	27.9
	EEIL	60.4	56.1	52.3	47.1	45.0	40.5
	LUCIR	63.8	62.4	59.1	49.1	48.5	42.8
Regularization $K = 0$	LwF-MC	45.9	27.4	20.1	29.1	23.1	17.4
	EWC	24.5	21.2	15.9	18.8	15.8	12.4
Prototype $K = 1$	PASS	63.5	61.8	58.1	49.6	47.3	42.1
	IL2A	66.0	60.3	57.9	47.3	44.7	40.0
	FeTril	<u>66.3</u>	<u>65.2</u>	61.5	54.8	53.1	52.2
	GBM_2^H	64.5	64.3	61.1	53.4	52.7	52.2
	GBM_1^T	66.0	65.8	64.3	53.9	53.2	52.8
Multiprototype $K = 8$	GBM_2^H	65.2	65.0	61.7	54.0	<u>53.3</u>	52.9
	GBM_1^T	67.1	67.0	64.4	<u>54.6</u>	54.0	53.7

Compared Methods. We compare GBM with the two proposed binarizers, p -bits thermometer, GBM_p^T , and f -heaviside, GBM_f^H , to EFCIL methods: LwF-MC (Li & Hoiem, 2017), EWC (Kirkpatrick et al., 2017) which are memory-free and PASS (Zhu et al., 2021b), IL2A Zhu et al. (2021a), FeTrIL Petit et al. (2023) which store class-prototypes. We also compare to state-of-the-art exemplars-based method iCaRL-CNN Rebuffi et al. (2017), EEIL Castro et al. (2018) and LUCIR Hou et al. (2019). Finally, we evaluate a Joint baseline, jointly trained on the entire dataset at once, to serve as an upper performance bound. It is worth mentioning that all compared methods are designed as real-valued models and replay exemplars or pseudo-exemplars are continuous feature vectors.

Metrics. We report the average accuracy. Average accuracy is the average of the $T + 1$ task accuracy (test set) on all the classes that have already been learned so far. We also consider final train and test accuracy to better understand the influence of embedding binarizer blocks.

Training and hyperparameters. Following the training protocol in Petit et al. (2023), we train ResNet-18 from scratch using a batch size of 128 and the SGD optimizer with a momentum of 0.9. The initial learning rate is 0.1, exponentially decaying by a factor of 0.1 every 50 epochs, for a total of 160 epochs. Data augmentation includes random horizontal flips, 4-pixel translations, and random contrast adjustments. For BMM computation, we use $N_{init} = 5$ and $N_{iter} = 3$ for warm-up, with $\epsilon = 10^{-3}$ and a maximum of $n_{max} = 10$ steps for stopping criteria. Results are averaged over 3 runs.

4.2. Results and discussion

Comparison with the State-of-the-art. Table 1 reports the experimental results on CIFAR100 and TinyImageNet for the three incremental settings. The results indicate that our 8-prototypes GBM_1^T has consistently better performance than state-of-the-art, on nearly every configuration. Compared to the second-best method, FeTrIL, GBM's accuracy is +1.5% higher, on the most challenging case of TinyImageNet with $T = 20$. Moreover our method maintains high final performance when the number of tasks increases, with a relative decrease of 1.6% on TinyImageNet between $T = 5$ and $T = 20$, compared to a relative 4.7% for FeTrIL.

Furthermore our 1-prototype GBM_1^T exhibits the best performance among single-prototype approaches on $T = 10$ and $T = 20$. For $T = 5$, GBM_1^T remained at -0.9% of SOTA FeTrIL on TinyImageNet while hav-

ing features precision on 1-bit compared 32-bits floating-point. Finally, Fig. 4a) shows that the 8-prototype GBM_1^T outperforms other methods on every task.

Memory footprint. A low memory footprint is a desirable property of incremental learning algorithms in embedded or memory-constrained environments Hayes and Kanan (2022), Petit et al. (2023), Rebuffi et al. (2017). All exemplar-free CIL methods must store a compact representation of past classes to alleviate catastrophic forgetting. Distillation-based methods such as PASS Zhu et al. (2021b) and IL2A Zhu et al. (2021a) require keeping both the previous and current models during incremental updates, effectively doubling memory usage. For a ResNet-18 backbone, this amounts to approximately 22.8M parameters, which is prohibitive for embedded deployment. Fixed feature extractor approaches, such as FeTrIL Petit et al. (2023), store a single frozen backbone (11.4M parameters) and maintain one centroid per class in a D -dimensional feature space, adding a negligible overhead of 512C parameters for C classes. IL2A Zhu et al. (2021a), in addition to class means, stores a covariance matrix of size $D \times D$ for each class, leading to a prohibitive $512^2 C$ parameter footprint when using ResNet-18 features.

In GBM, each class is modeled as a mixture of K Bernoulli prototypes, each represented in floating-point precision as a D -dimensional probability vector. Thus, the memory footprint for one prototype is equivalent to that of FeTrIL, and the total storage scales linearly with the number of mixture components, i.e., $K \times D \times C$. This formulation allows for a controllable trade-off between memory cost and classification performance by adjusting K . Moreover, since prototype components are probability values bounded between 0 and 1, they can be uniformly quantized into integer representations without significant loss of information. This quantization aspect, further evaluated in Section 5, offers an additional lever to reduce the memory footprint while preserving model accuracy.

Prototype computation complexity. The computational complexity of prototype generation varies across CIL methods. PASS Zhu et al. (2021b) and IL2A Zhu et al. (2021a) both approximate class distributions through Gaussian modeling. In PASS, class prototypes are obtained as the mean feature vector of all samples belonging to each class, leading to a linear complexity $\mathcal{O}(ND)$, where N denotes the number of samples and D the feature dimensionality. Although covariance matrices are not stored, they must be temporarily computed to estimate the class-wise variance used for defining the generation radius, which introduces an additional $\mathcal{O}(ND^2)$ cost. IL2A explicitly estimates and stores both the mean vector and the covariance matrix for each class, resulting in a total complexity of $\mathcal{O}(ND^2)$ and a prohibitive cost for high-dimensional features (e.g., $D = 512$ for ResNet-18). FeTrIL Petit et al. (2023) similarly computes class centroids as the empirical mean of frozen features, also with complexity $\mathcal{O}(ND)$, and thus remains computationally efficient.

In contrast, GBM estimates class prototypes as mixtures of K Bernoulli models in a binary latent space of dimension D . Each mixture component is learned through an Expectation-Maximization (EM) algorithm with per-iteration complexity $\mathcal{O}(NKD)$. While iterative, convergence is typically reached in a few iterations (< 10 in practice), keeping the total cost moderate. The binary representation space and small mixture size ($K \in \{1, 8\}$) further contribute to the computational efficiency of the approach. Regarding prototype-based sample generation, all methods, PASS, FeTrIL, IL2A, and GBM, require comparable computational effort, as this step consists of sampling or decoding from stored prototype statistics once they have been estimated.

Sensitivity to quantization. Table 2 reports train and test accuracy on the initial task and the final incremental task of CIFAR100, $T = 10$ for different size of binary embedding, with $K = 8$ prototypes. For GBM^T increasing the number of bits marginally improves test performance of the initial task by making F more expressive. On the contrary, surprisingly, a higher precision results in lower train and test accuracy on the final task. One reason can be that a lower-precision embedding regularizes the training and offers more transferable features to future incremental tasks, thus easing the optimization (train set) and enhancing generalisation (test set). For GBM^H , the initial and final performance is

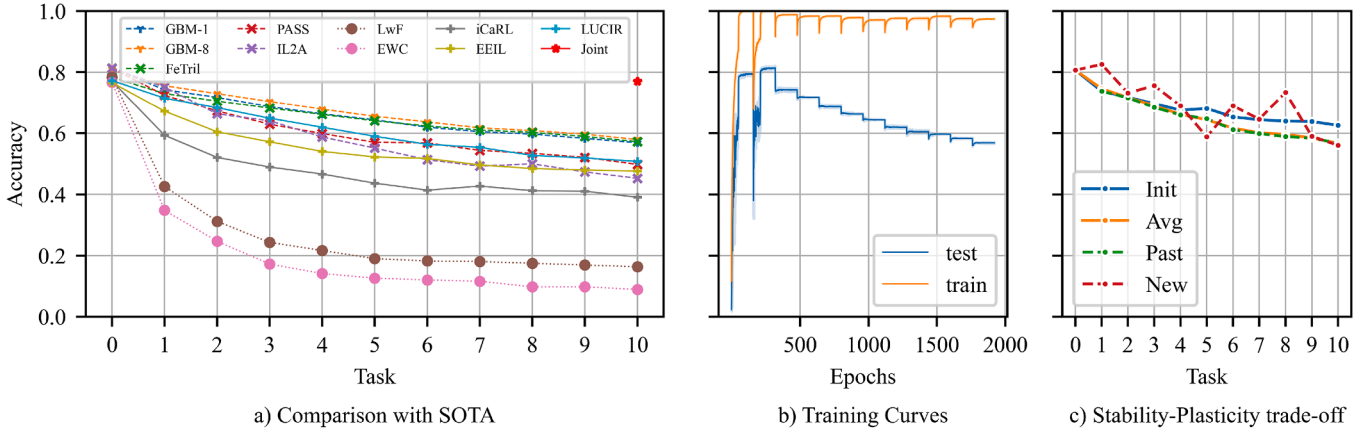


Fig. 4. Incremental performance of GBM^T on CIFAR100, $T=10$. a) Comparison against state-of-the-art. b) Training curves on test and training sets. c) Stability-plasticity trade-off, comparing seen class accuracy (Avg) on a subset of New, Past, Initial (Init) classes.

Table 2

Investigation on the binarization method applied to Resnet-18. Results are on CIFAR100, $T=10$, $K=8$ and for both Heaviside (GBM^H) and Thermometer (GBM^T) binarizations.

D	p or f	method	Task Init (0)		Task Final (10)	
			Train	Test	Train	Test
512	1	GBM^T	100.0	81.1	76.1	57.9
		GBM^H	100.0	81.1	73.5	54.9
1024	2	GBM^T	100.0	81.3	75.1	57.7
		GBM^H	100.0	81.0	73.6	54.6
2048	4	GBM^T	100.0	81.5	74.5	56.7
		GBM^H	100.0	80.7	73.6	54.3

Table 3

Ablation study on the STE training phase during the initial training with Thermometer (GBM^T) on CIFAR100 ($T=10$).

p	test acc. w/o STE phase	test acc. w/ STE phase
1	69.9	81.47
2	75.0	81.24
4	76.4	81.18
ResNet-18 He et al. (2016)		81.54

not affected by the embedding size, with less than 1% difference in the performance. We report results for GBM_2^H , GBM_1^T with $K=1$ and $K=8$ on CIFAR100 and TinyImageNet benchmark Table 1. GBM^T always yields better performance than GBM^H . Adding a projection to learn binary features is not beneficial. Fig. 4b) presents the training curves during all incremental retrainings for GBM_1^T , $K=8$, our best configuration. The train accuracy, computed on training batches (samples and pseudo-exemplars), nearly always reaches 100% accuracy. It suggests that the embedding binarizer does not prevent F and G to be correctly optimized, despite quantization.

Influence of training with STE. Table 3 reports the initial test accuracy for $p=1,2,4$ with and without a second training phase with STE, in GBM^T . The results indicate that this second training phase is necessary so that a ResNet-18 with binarization reaches the same performance as without binarization, even for low-precision thermocodes on 1-bit ($p=1$). F can effectively be trained to mitigate quantization error and learn feature clipping.

Influence of BMM initialization. Table 4 presents the results for the combinations of the EM-algorithm's μ initialization and π update, for $K=1,2,4$. When the number of prototype augments, centroid initialization is preferable to a random one. Even if intra-class clusters are

Table 4

Influence of BMM hyper-parameters on the final average accuracy. Results are reported for GBM_1^T on CIFAR100 ($T=5$).

μ init.	π update	$K=1$	$K=2$	$K=4$
random	fixed	65.7	66.0	66.9
random	trainable	66.2	67.1	67.3
centroid	fixed	66.1	66.7	67.0
centroid	trainable	66.0	67.4	67.5

noticeable (Fig. 5) they spread closely to the class centroid. Making π trainable improves final average accuracy over having them fixed, meaning that accurate mixing coefficients boost performance. For each K , the relative difference among all cases never exceeds 1.5%. Table 4 therefore demonstrates that for such a CIL scenario, our method remains robust to a trainable π with a random μ initialization.

Stability-Plasticity trade-off. CIL methods should preferably ensure similar classification performance for every class, new and past. To reach this goal, the number of pseudo-exemplars generated at each iteration are so that each batch contains an equal number of elements per class. We empirically explore plasticity and stability by evaluating the accuracy on the initial, new, past, and all seen classes in Fig. 4. Several observations can be made. New classes are better classified than past classes (except on task 5). Even with a fixed F , the classifier can adapt to new classes. Nevertheless, the average relative difference between past and new accuracy is 4.5%, indicating a good trade-off between stability and plasticity. Finally, initial classes are better classified than past classes, hence freezing the encoder slightly prioritize (< 6%) the retention of initial classes.

Feature Visualization. Fig. 5 illustrates the generation of pseudo-exemplars for GBM^T , with t-SNE Van der Maaten and Hinton (2008) transformations. For a single prototype, $K=1$, on both the initial and incremental classes, pseudo-exemplars effectively approximate the sample distribution near by the centroid; however, farther away from the centroid, pseudo-exemplars do not completely cover the sample distribution, particularly when the thermocode precision is higher, i.e. $p=4$. This may be due to the fact that the thermometer coding may introduce feature correlations, and a BMM with a single prototype fails to represent this correlation, as illustrated in Fig. 5c). Note that the Frobenius distance between the correlation matrices of samples and pseudo-exemplars remains large. In contrast, with $K=8$, pseudo-exemplars more precisely capture sub-modalities within the class distribution, even when $p=4$. This finding is further supported by a decrease in Frobenius distance of 33%. Moreover, when comparing Fig. 5a) and b) we observe that the inter-class distance among incremental task classes is

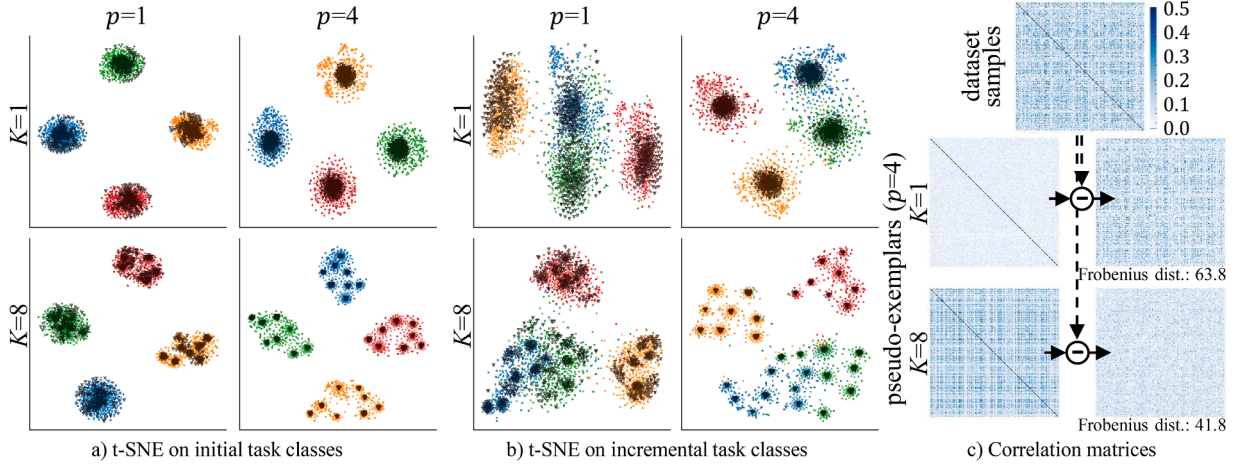


Fig. 5. Embedding visualization on CIFAR100 with GBM_1^T . a-b) t-SNE Van der Maaten and Hinton (2008) on 4 classes, samples (light shade) and generated pseudo-exemplars (dark shade). c) Absolute difference between the correlation matrices of exemplars (dataset samples) and pseudo-exemplars.

qualitatively smaller than that of the initial task classes. Consequently, employing multiple prototypes is advantageous, as it eases the generation of pseudo-exemplars closer to the decision boundaries.

5. Evaluation on BNN architectures

The literature on CIL for BNN addresses two types of networks: (1) **Hybrid BNN**, for TinyML applications, where the model size is $\sim 10\text{Mb}$ and full-precision operations are still included Bannink et al. (2021); (2) **Fully BNN**, for new hardware technologies and architecture design, where the model size is $\sim 1\text{Mb}$ and binary-only arithmetic is permitted during inference. Here we show the applicability of our GBM in both distinct model types. The accuracy and memory requirements, $\mathcal{M}$, are compared only against LR approaches that scale to large datasets Basso-Bert et al. (2024), Lorenzo et al. (2024). The memory size of the GBM prototypes depends on the number of prototypes K , prototype precision q , embedding size D , and total number of classes n_c , with $\mathcal{M}(\text{GBM}) = K \times D \times n_c \times q$ bits. For binary LR, the memory size depends on the number of latent exemplars E , with $\mathcal{M}(\text{LR}) = E \times D \times n_c \times 1$ bits.

5.1. Experimental results on hybrid BNN

We compare to the state-of-the-art LR + CWR* method, with their network and dataset settings Lorenzo et al. (2024). As in LR + CWR*, in GBM the QuickNetLarge backbone Bannink et al. (2021) is pre-trained on the ImageNet Deng et al. (2009) dataset. F ends at the output of the *quant_conv2d_30* layer, with an output dimension of $D = 12544$ features, and the classifier, $\mathcal{G}$, consists of the next 14 layers. The benchmark is CORE50 in the *NC scenario* Lomonaco and Maltoni (2017): 10 domestic objects classes learned in 9 incremental classification tasks starting with 2 objects.

Table 5 reports final accuracy for GBM and LR + CWR* for different sizes of memory. With $K=1$, GBM gives +3.1% higher final accuracy than LR + CWR* with the largest buffer, and requires $\times 4.7$ less memory space. Increasing the number of prototypes slightly increases the final accuracy by an absolute +0.5%.

5.2. Experimental results on fully BNN

Prior incremental learning work is less mature on Fully BNNs. We compare GBM with LR Basso-Bert et al. (2024) on their fully- binary VGG-like network of $\sim 1\text{Mb}$, on the CIFAR100 dataset.

An important goal is to minimize memory requirements for CIL with BNN. Fig. 6 shows the memory size required by CIL and the average incremental accuracy on CIFAR100, $T = 10$. Three cases are compared:

Table 5

Comparison to Lorenzo et al. (2024) on CORE50 NC benchmark.

Method		Memory size ($\mathcal{M}$)	Final accuracy
LR + CWR* Lorenzo et al. (2024)	$E = 75$	9.4 Mb	69.5%
	$E = 100$	12.5 Mb	70.6%
	$E = 150$	18.8 Mb	70.8%
GBM	$K = 1$	4.0 Mb	73.9%
	$K = 4$	16.0 Mb	74.2%
	$K = 8$	32.0 Mb	74.6%

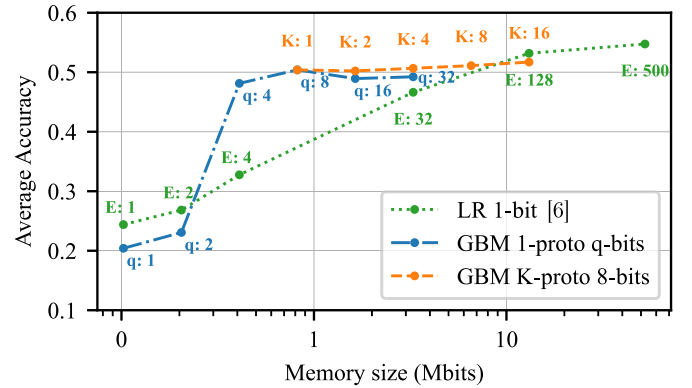


Fig. 6. Fully BNN on CIFAR-100, LR and GBM: average incremental accuracy versus required memory size.

(1) LR with E ranging from 1 to 500, (2) GBM $K=1$ with q ranging from 1 to 32 to assess prototype quantization, and (3) GBM $q=8$ with K ranging from 1 to 16. GBM prototypes with $q \leq 2$ cannot properly model the embeddings distribution, therefore performing worse than LR. Nevertheless, for $q \geq 3$, GBM outperforms LR with a peak at $q=8$. To achieve the same accuracy (50.4%), LR requires a $\times 10$ larger replay buffer. With $q=8$ performance can still be improved up to an additional +2% by increasing the number of prototypes from 1 to 16. Prototypes quantization thus increases performance while limiting the memory required ($\leq 10\text{Mb}$).

6. Limitations and possible extensions

As Petril Petit et al. (2023), the final accuracy of our approach relies on a fixed F , making CIL performance dependent on the initial training

on D_0 . GBM can be extended to a trainable $\mathcal{F}$, constrained with knowledge distillation [Zhu et al. \(2021a,b\)](#).

Furthermore, unlike standard CIL scenario benchmarks, in real-world applications, new classes might not emerge in large, discrete task datasets but as a new knowledge acquired sample-by-sample in an online manner [Hayes and Kanan \(2022\)](#). Nevertheless, online versions of the EM-algorithm [Cappé and Moulines \(2009\)](#) could extend the GBM to Online-CIL [Aljundi et al. \(2019\)](#) and Few-Shot CIL [Tao et al. \(2020\)](#).

On ResNet-18, we employ a linear classifier, which constrains the classes embedding to be linearly separable. Our multi-prototype approach offers the potential to approximate non-linearly separable classes distribution. Thus GBM could further benefit from more complex classifiers.

Finally, the assumption that binary latent representations follow Bernoulli mixture distributions may not hold in more complex or non-visual modalities, where latent structures exhibit higher-order dependencies. In such cases, alternative probabilistic formulations or richer latent priors could be explored to improve representational fidelity.

7. Conclusion

This paper introduces the Generative Binary Memory (GBM) method to enable pseudo-replay in Binary Neural Networks (BNNs), relying on Bernoulli Mixture Model (BMM). In addition, we demonstrate its applicability to real-valued networks, through the additional use of an embedding binarizer, achieving higher average accuracy on challenging benchmarks, such as TinyImageNet and CORE50, compared to state-of-the-art pseudo-replay and replay methods. Furthermore, GBM alleviates catastrophic forgetting in both large BNNs and fully binarized models, while providing superior incremental performance with a lower memory footprint, when compared to prior methods.

CRedit authorship contribution statement

Yanis Basso-Bert: Conceptualization, Data curation, Formal analysis, Investigation, Methodology, Software, Validation, Visualization, Writing – original draft, Writing – review & editing; **William Guicquero:** Conceptualization, Funding acquisition, Investigation, Methodology, Project administration, Software, Supervision, Writing – original draft, Writing – review & editing; **Anca Molnos:** Conceptualization, Funding acquisition, Project administration, Resources, Software, Supervision, Writing – original draft, Writing – review & editing; **Romain Lemaire:** Investigation, Methodology, Project administration, Software, Supervision, Writing – review & editing; **Antoine Dupret:** Supervision, Writing – review & editing.

Declaration of competing interest

None

Acknowledgments

This work is part of the IPCEI Microelectronics and Connectivity and was supported by the French Public Authorities within the frame of France 2030.

References

Alalyan, F., Zamzami, N., & Bouguila, N. (2020). A hybrid approach based on SVM and bernoulli mixture model for binary vectors classification. In *2020 IEEE International conference on systems, man, and cybernetics (SMC)* (pp. 1155–1160). IEEE.

Aljundi, R., Lin, M., Goujaud, B., & Bengio, Y. (2019). Gradient based sample selection for online continual learning. *Advances in Neural Information Processing Systems*, 32.

Amato, G., Falchi, F., Rabitti, F., & Vadicamo, L. (2016). Combining fisher vector and convolutional neural networks for image retrieval. In *Ir*.

Amato, G., Falchi, F., & Vadicamo, L. (2018). Aggregating binary local descriptors for image retrieval. *Multimedia Tools and Applications*, 77, 5385–5415.

Bannink, T., Hillier, A., Geiger, L., de Bruin, T., Overweel, L., Neeven, J., & Helwegen, K. (2021). Larq compute engine: Design, benchmark and deploy state-of-the-art binarized neural networks. *Proceedings of Machine Learning and Systems*, 3, 680–695.

Basso-Bert, Y., Guicquero, W., Molnos, A., Lemaire, R., & Dupret, A. (2024). On class-incremental learning for fully binarized convolutional neural networks. In *2024 IEEE International symposium on circuits and systems (ISCAS)* (pp. 1–5). IEEE.

Bengio, Y., Léonard, N., & Courville, A. (2013). Estimating or propagating gradients through stochastic neurons for conditional computation. arXiv:1308.3432.

Bishop, C. M., & Nasrabadi, N. M. (2006). *Pattern recognition and machine learning* (vol. 4). Springer.

Buckman, J., Roy, A., Raffel, C., & Goodfellow, I. (2018). Thermometer encoding: One hot way to resist adversarial examples. In *International conference on learning representations*.

Cappé, O., & Moulines, E. (2009). On-line expectation–maximization algorithm for latent data models. *Journal of the Royal Statistical Society Series B: Statistical Methodology*, 71(3), 593–613.

Castro, F. M., Marín-Jiménez, M. J., Guil, N., Schmid, C., & Alahari, K. (2018). End-to-end incremental learning. In *Proceedings of the european conference on computer vision (ECCV)* (pp. 233–248).

Deng, J., Dong, W., Socher, R., Li, L.-J., Li, K., & Fei-Fei, L. (2009). Imagenet: A large-scale hierarchical image database. In *2009 IEEE Conference on computer vision and pattern recognition* (pp. 248–255). Ieee.

Gimenez, A., Andrés-Ferrer, J., & Juan, A. (2014). Discriminative bernoulli HMMs for isolated handwritten word recognition. *Pattern Recognition Letters*, 35, 157–168.

Gou, J., Yu, B., Maybank, S. J., & Tao, D. (2021). Knowledge distillation: A survey. *International Journal of Computer Vision*, 129(6), 1789–1819.

Hadsell, R., Rao, D., Rusu, A. A., & Pascanu, R. (2020). Embracing change: Continual learning in deep neural networks. *Trends in Cognitive Sciences*, 24(12), 1028–1040.

Hayes, T. L., & Kanan, C. (2022). Online continual learning for embedded devices. In *Conference on lifelong learning agents (colLAs)*.

He, K., Zhang, X., Ren, S., & Sun, J. (2016). Deep residual learning for image recognition. In *Proceedings of the IEEE conference on computer vision and pattern recognition* (pp. 770–778).

Hou, S., Pan, X., Loy, C. C., Wang, Z., & Lin, D. (2019). Learning a unified classifier incrementally via rebalancing. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition* (pp. 831–839).

Hubara, I., Courbariaux, M., Soudry, D., El-Yaniv, R., & Bengio, Y. (2016). Binarized neural networks. *Advances in Neural Information Processing Systems*, 29.

Juan, A., García-Hernández, J., & Vidal, E. (2004). Em initialisation for bernoulli mixture learning. In *Joint IAPR international workshops on statistical techniques in pattern recognition (SPR) and structural and syntactic pattern recognition (SSPR)* (pp. 635–643). Springer.

Kirkpatrick, J., Pascanu, R., Rabinowitz, N., Veness, J., Desjardins, G., Rusu, A. A., Milan, K., Quan, J., Ramalho, T., Grabska-Barwinska, A. et al. (2017). Overcoming catastrophic forgetting in neural networks. *Proceedings of the National Academy of Sciences*, 114(13), 3521–3526.

Korycki, L., & Krawczyk, B. (2024). Class-incremental mixture of gaussians for deep continual learning. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition* (pp. 4097–4106).

Krizhevsky, A., & Hinton, G. (2009). Learning multiple layers of features from tiny images. Technical Report 0. University of Toronto Toronto, Ontario. <https://www.cs.toronto.edu/~kriz/learning-features-2009-TR.pdf>.

Kulkarni, U., Hosamani, A. S., Masur, A. S., Hegde, S., Vernekar, G. R., & Chandana, K. S. (2022). A survey on quantization methods for optimization of deep neural networks. In *2022 International conference on automation, computing and renewable systems (ICACRS)* (pp. 827–834). IEEE.

Laborieux, A., Ernout, M., Hirtzlin, T., & Querlioz, D. (2021). Synaptic metaplasticity in binarized neural networks. *Nature Communications*, 12(1), 2549.

Lazarsfeld, P. F., & Henry, N. W. (1968). *Latent Structure Analysis*. Houghton, Mifflin. <https://books.google.fr/books?id=jsFPAQAIAAJ>.

Le, Y., & Yang, X. (2015). Tiny imagenet visual recognition challenge. *CS 231N*, 7(7), 3.

LeCun, Y., Bottou, L., Bengio, Y., & Haffner, P. (1998). Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11), 2278–2324.

Li, C., Wang, B., Pavlu, V., & Aslam, J. (2016). Conditional bernoulli mixtures for multi-label classification. In *International conference on machine learning* (pp. 2482–2491). PMLR.

Li, Z., & Hoiem, D. (2017). Learning without forgetting. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 40(12), 2935–2947.

Liu, Z., Shen, Z., Savvides, M., & Cheng, K.-T. (2020). Reactnet: Towards precise binary neural network with generalized activation functions. In *Computer vision–ECCV 2020: 16th european conference, glasgow, UK, august 23–28, 2020, proceedings, part XIV 16* (pp. 143–159). Springer.

Liu, Z., Wu, B., Luo, W., Yang, X., Liu, W., & Cheng, K.-T. (2018). Bi-real net: Enhancing the performance of 1-bit cnns with improved representational capability and advanced training algorithm. In *Proceedings of the european conference on computer vision (ECCV)* (pp. 722–737).

Loaiza-Ganem, G., & Cunningham, J. P. (2019). The continuous bernoulli: Fixing a pervasive error in variational autoencoders. *Advances in Neural Information Processing Systems*, 32.

Lomonaco, V., & Maltoni, D. (2017). Core50: A new dataset and benchmark for continuous object recognition. In *Conference on robot learning* (pp. 17–26). PMLR.

Lorenzo, V., Davide, M., Guido, B., & Stefano, S. (2024). Enabling on-device continual learning with binary neural networks and latent replay. In *Proceedings of the 19th international joint conference on computer vision, imaging and computer graphics theory and applications - volume 2: VISAPP* (pp. 25–36). INSTICC SciTePress. <https://doi.org/10.5220/0012269000003660>

- Ma, R., Qiao, G., Liu, Y., Meng, L., Ning, N., Liu, Y., & Hu, S. (2024). A&b BNN: Add&bit-operation-only hardware-friendly binary neural network. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition* (pp. 5704–5713).
- Van der Maaten, L., & Hinton, G. (2008). Visualizing data using t-SNE. *Journal of Machine Learning Research*, 9(11).
- McCloskey, M., & Cohen, N. J. (1989). Catastrophic interference in connectionist networks: the sequential learning problem. In *Psychology of learning and motivation* (pp. 109–165). Elsevier (vol. 24).
- Najafi, A., Motahari, A., & Rabiee, H. R. (2019). Reliable clustering of bernoulli mixture models. <https://arxiv.org/abs/1710.02101>.
- Nielsen, F. (2016). Introduction to HPC with MPI for Data Science. Springer.
- Pellegrini, L., Graffieti, G., Lomonaco, V., & Maltoni, D. (2020). Latent replay for real-time continual learning. In *2020 IEEE/RSJ International conference on intelligent robots and systems (IROS)* (pp. 10203–10209). IEEE.
- Petit, G., Popescu, A., Schindler, H., Picard, D., & Delezoide, B. (2023). Fetril: Feature translation for exemplar-free class-incremental learning. In *Proceedings of the IEEE/CVF winter conference on applications of computer vision* (pp. 3911–3920).
- Pföhl, B., & Gepperth, A. (2021). Overcoming catastrophic forgetting with gaussian mixture replay. In *2021 International joint conference on neural networks (IJCNN)* (pp. 1–9). IEEE.
- Ravaglia, L., Rusci, M., Nadalini, D., Capotondi, A., Conti, F., & Benini, L. (2021). A tinyml platform for on-device continual learning with quantized latent replays. *IEEE Journal on Emerging and Selected Topics in Circuits and Systems*, 11(4), 789–802.
- Rebuffi, S.-A., Kolesnikov, A., Sperl, G., & Lampert, C. H. (2017). Icarl: Incremental classifier and representation learning. In *Proceedings of the IEEE conference on computer vision and pattern recognition* (pp. 2001–2010).
- Saeed, M., Javed, K., & Babri, H. A. (2013). Machine learning using bernoulli mixture models: clustering, rule extraction and dimensionality reduction. *Neurocomputing*, 119, 366–374.
- Tao, X., Hong, X., Chang, X., Dong, S., Wei, X., & Gong, Y. (2020). Few-shot class-incremental learning. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition* (pp. 12183–12192).
- Wang, L., Zhang, X., Su, H., & Zhu, J. (2024). A comprehensive survey of continual learning: Theory, method and application. *IEEE Transactions on Pattern Analysis and Machine Intelligence*.
- Wang, L., Zhang, X., Yang, K., Yu, L., Li, C., Lanqing, H., Zhang, S., Li, Z., Zhong, Y., & Zhu, J. (2021). Memory replay with data compression for continual learning. In *International conference on learning representations*.
- Wu, S., Diao, E., Ding, J., & Tarokh, V. (2019). Deep clustering of compressed variational embeddings. [arXiv:1910.10341](https://arxiv.org/abs/1910.10341).
- Yan, S., Xie, J., & He, X. (2021). Der: Dynamically expandable representation for class incremental learning. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition* (pp. 3014–3023).
- Yang, L., & Wu, W. (2020). Multiple-instance learning based on bernoulli mixture model. In *Journal of physics: Conference series* (p. 032071). IOP Publishing (vol. 1650).
- Yuan, C., & Agaian, S. S. (2023). A comprehensive review of binary neural network. *Artificial Intelligence Review*, 56(11), 12949–13013.
- Zhu, F., Cheng, Z., Zhang, X.-Y., & Liu, C.-I. (2021a). Class-incremental learning via dual augmentation. *Advances in Neural Information Processing Systems*, 34, 14306–14318.
- Zhu, F., Zhang, X.-Y., Wang, C., Yin, F., & Liu, C.-I. (2021b). Prototype augmentation and self-supervision for incremental learning. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition* (pp. 5871–5880).